

Engineering Reflection Questions

1. Why should source profiling occur before implementation of ingestion?

Profiling enables the identification of data structures, nullability, data types, volume sizes, and anomalies proactively. This approach prevents runtime failures by informing the design of robust data contracts, suitable schemas, and precise error-handling logic prior to the development of code.

2. What is the difference between source event time/updated_at and ingestion time?

The source event time (updated_at) signifies the moment a record was either created or modified at the source system. Conversely, the ingestion time (_ingested_at) records the precise operational timestamp when the pipeline retrieved and stored the raw data locally.

3. Why is event_id alone insufficient to decide which duplicate API record to keep in this exercise?

Entities can be updated multiple times, resulting in multiple records sharing the same event_id. Relying solely on the ID without verifying the updated_at timestamp risks maintaining an outdated state instead of the most recent revision.

4. Why must the watermark state advance only after successful persistence?

By advancing the watermark prior to successful raw writes, a window of opportunity for data loss is created. Should the pipeline encounter a crash during the middle of a write operation, the advanced watermark would ensure that subsequent runs permanently skip those failed records.

5. What limitation does updated_after > watermark have when multiple source records can share exactly the same timestamp?

A strict inequality filter (>) can cause records that share the precise timestamp boundary as the watermark to be excluded if pagination divides a batch, resulting in silent data omission.

6. How is duplicate prevention related to idempotency?

Duplicate prevention guarantees that executing the pipeline multiple times over the same data results in identical, clean outcomes, eliminating the duplication of records, which is the fundamental principle underlying pipeline idempotency.

7. Why should the raw area preserve source values instead of applying business transformations?

Preserving raw data ensures an immutable and faithful audit trail of the source. This guarantees that if downstream transformation logic undergoes changes or encounters failures, developers can consistently reprocess from the original raw data.

8. How could querying a production OLTP source for profiling or extraction degrade the application?

Excessive analytical queries, full table scans, or database locks can significantly drain valuable CPU memory and connection pool resources. Consequently, these actions directly introduce latency or downtime for live user-facing applications.

9. What would you change if the API had a rate limit of 60 requests per minute?

To handle a strict rate limit, the pipeline would need rate limiting and retry mechanisms, such as introducing time delays (time.sleep()) between page requests and implementing an exponential backoff strategy, or using token bucket algorithms to gracefully pause and resume extraction when HTTP 429 (Too Many Requests) errors occur.

10. How would you extend this pipeline from a local raw area to PostgreSQL while preserving rerun safety?

Rerun safety and idempotency would be maintained by leveraging database-level unique constraints and upsert operations and tracking file ingestion manifests inside a metadata table with unique SHA-256 constraints, rather than relying solely on local file checks.